Práctica 2:

Curso: Algoritmos y Estructuras de Datos
Tema: Implementaciones y análisis de Bubble, Selection, Insertion y Shell Sort
Duración estimada: 1 hora

1. OBJETIVO

- Implementar en C++ y Python los algoritmos Bubble, Selection, Insertion y Shell Sort.
- Analizar eficiencia temporal (mejor, promedio, peor casos), eficiencia en swaps/comparaciones y estabilidad.
- Evaluar diferencias prácticas entre los métodos, alineados con el marco teórico.

2. MARCO TEÓRICO

Algoritmos de ordenamiento básicos: Bubble, Selection, Insertion y Shell Sort

El estudio de los algoritmos de ordenamiento es fundamental en la formación en ciencias de la computación, no solo por su aplicabilidad directa en problemas de organización de datos, sino porque constituyen la base conceptual para comprender algoritmos avanzados de búsqueda y estructuras de datos complejas.

Bubble Sort, aunque ineficiente en términos de complejidad O(n2)O(n^2)O(n2), se utiliza aún con fines educativos para ilustrar conceptos básicos de intercambio y comparación (Astrachan, 2003). Investigaciones recientes incluso proponen variantes que alcanzan un mejor rendimiento en el mejor caso (Ω(n)\Omega(n)Ω(n)) (Rahmani, 2022).

Selection Sort presenta un número constante de comparaciones (O(n2)O(n^2)O(n2)), lo que lo hace predecible, aunque poco eficiente. Aun así, su bajo número de intercambios lo convierte en una opción útil cuando las operaciones de swap son costosas (Research Publish, s. f.).

Insertion Sort es especialmente eficiente en listas pequeñas o casi ordenadas, donde puede alcanzar O(n)O(n)O(n) en el mejor caso. Es estable y adaptable, lo que explica por qué se emplea como subrutina en algoritmos más complejos como Quicksort o Timsort (Machkasova, 2011; Silas et al., 2016).

Shell Sort, por su parte, representa una optimización del Insertion Sort al aplicar la comparación por intervalos decrecientes (gaps). Su rendimiento depende de la secuencia de gaps utilizada, alcanzando en algunos casos O(nlog2n)O(n \log^2 n)O(nlog2n). En arreglos de tamaño medio a grande, puede superar a Insertion Sort en rendimiento práctico (Silas et al., 2016).

Finalmente, estudios recientes han vinculado la **eficiencia temporal** con la **eficiencia energética**, demostrando que algoritmos más costosos computacionalmente, como Bubble Sort, también implican mayor consumo energético (Carter et al., 2023). Además, se ha demostrado que para arreglos muy pequeños existen alternativas modernas como

las *sorting networks*, que superan significativamente a Insertion Sort en rendimiento (Bingmann et al., 2020).

3. PROCEDIMIENTO

a) Implementación (C++ y Python)

C++:

```
#include <iostream>
#include <vector>
#include <chrono>
using namespace std;
using Clock = chrono::high_resolution_clock;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool swapped;
    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx]) minIdx = j;
        swap(arr[i], arr[minIdx]);
    }
}

void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void shellSort(vector<int>& arr) {
    int n = arr.size();
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
```

```
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
                arr[j] = arr[j - gap];
            arr[j] = temp;
        }
    }
}

void test(vector<int> data, void(*func)(vector<int>&), const string&
name) {
    auto arr = data;
    auto t1 = Clock::now();
    func(arr);
    auto t2 = Clock::now();
    cout << name << " took "
         << chrono::duration_cast<chrono::microseconds>(t2 -
t1).count()
         << " µs\n";
}

int main() {
    vector<int> sizes = {10, 100, 1000};
    for (int n : sizes) {
        vector<int> data(n);
        for (int i = 0; i < n; i++) data[i] = rand();
        cout << "--- Size: " << n << " ---\n";
        test(data, bubbleSort, "Bubble Sort");
        test(data, selectionSort, "Selection Sort");
        test(data, insertionSort, "Insertion Sort");
        test(data, shellSort, "Shell Sort");
    }
    return 0;
}
```

Python:

```
import random
import time

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break

def selection_sort(arr):
    for i in range(len(arr) - 1):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

def insertion_sort(arr):
```

```
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

def shell_sort(arr):
    n = len(arr)
    gap = n // 2
    while gap > 0:
        for i in range(gap, n):
            temp = arr[i]
            j = i
            while j >= gap and arr[j - gap] > temp:
                arr[j] = arr[j - gap]
                j -= gap
            arr[j] = temp
        gap //= 2

def test(data, func, name):
    arr = data.copy()
    start = time.time()
    func(arr)
    end = time.time()
    print(f"{name} took {(end - start)*1e6:.2f} µs")

for n in [10, 100, 1000]:
    data = [random.randint(0, 10000) for _ in range(n)]
    print(f"--- Size: {n} ---")
    test(data, bubble_sort, "Bubble Sort")
    test(data, selection_sort, "Selection Sort")
    test(data, insertion_sort, "Insertion Sort")
    test(data, shell_sort, "Shell Sort")
```

b) Pruebas de Rendimiento (20 min)

Ejecuta ambos programas para los tamaños: 10, 100 y 1000 elementos. Registra tiempos (µs), y si posible, compara con los conceptos teóricos.

c) Análisis (20 min)

- Contrasta resultados observados con las complejidades teóricas del marco.
- Nota que Insertion sort es óptimo cuando la lista ya tiene orden parcial, Self sort mejora sobre otros en eficiencia, y Bubble sort es el más lento en general.
- Discute estabilidad: Bubble e Insertion son estables; Selection y Shell no necesariamente.

4. TRABAJO DE INVESTIGACIÓN

Redacta un ensayo (~1 página) respondiendo:

- ¿Por qué seguimos estudiando algoritmos O(n2)O(n^2)O(n2) si existen más rápidos como Quicksort y Mergesort?
- ¿En qué escenarios son útiles métodos simples como Insertion o Selection (e.g., arreglos pequeños, casi ordenados, entornos con bajo overhead, sistemas embebidos)?
- Apóyate en las referencias APA proporcionadas.

5. RESULTADOS ESPERADOS

- Código completo y funcional en ambos lenguajes.
- Tabla comparativa de tiempos por algoritmo y tamaño.
- Análisis crítico: mejores casos observados, eficiencia práctica versus teórica, estabilidad, elección de algoritmo según contexto.

Referencias

1. Astrachan, O. L. (2003). *Bubble sort: An archaeological algorithmic analysis*. ACM SIGCSE Bulletin, 35(1), 1–5. https://users.cs.duke.edu/~ola/papers/bubble.pdf
2. Bingmann, T., Ehlers, T., & Sanders, P. (2020). *The Art of Practical Sorting Networks*. arXiv. https://arxiv.org/abs/2002.05599
3. Carter, N., Bird, J., & Fedorova, A. (2023). *The energy complexity of sorting*. arXiv. https://arxiv.org/abs/2311.07298
4. Machkasova, E. (2011). *Why insertion sort works best for small inputs*. University of Minnesota, Morris. https://personal.morris.umn.edu/~elenam/publications/sorting.pdf
5. Rahmani, R. (2022). *A novel variant of bubble sort with improved best-case performance*. Journal of Discrete Algorithms, 2(7), 30–32. https://www.sciencedirect.com/science/article/pii/S1546221822006518
6. Reddy, R. G., Anisha, P. R., Kumar Reddy, C. K. R., & Srinivasa Reddy, M. (2017). *An empirical study and analysis on sorting algorithms*. International Journal of Mechanical Engineering and Technology, 8(8), 488–498. https://iaeme.com/MasterAdmin/Journal_uploads/IJMET/VOLUME_8_ISSUE_8/IJMET_08_08_055.pdf
7. Research Publish. (s. f.). *Performance comparison of sorting algorithms on the basis of complexity*. Research Publish Journals. https://www.researchpublish.com/upload/book/Performance%20Comparison%20of%20Sorting%20Algorithms%20On%20The%20Basis%20Of%20Complexity-542.pdf
8. Silas Faki, M., Mu'azu, M. B., & Abdu, M. (2016). *An empirical analysis of internal sorting algorithms*. Journal of Advances in Mathematics and Computer Science, 16(2), 1–12. https://journaljamcs.com/index.php/JAMCS/article/view/283
9. Global Scientific Journal. (2021). *Performance evaluation of sorting techniques*. GSJ, 9(4), 103–112. https://www.globalscientificjournal.com/researchpaper/PERFORMANCE_EVALUATION_OF_SORTING_TECHNIQUES.pdf
10. Stack Overflow Community. (2017). *Empirical analysis of merge and insertion sort difficulties*. Stack Overflow.

https://stackoverflow.com/questions/45428894/empirical-analysis-of-merge-and-insertion-sort-difficulties